

From idea to merged PR

Seven steps in five phases: Requirements, Plan, Design, Build, Implement. Each step is one Claude command in its own session. It produces one artifact, or for step 3 one set of issues, and waits for your explicit approval before the next step will run. Review and merge stay with you.

● Local Claude session

● Cloud Claude session

● You (approve, review, merge)

● GitHub repo / project board

Where the card sits on the board

Status only moves forward, and only when something about the code changes. Labels show whose hands the item is in. Ideas in steps 1–2 have no card yet.

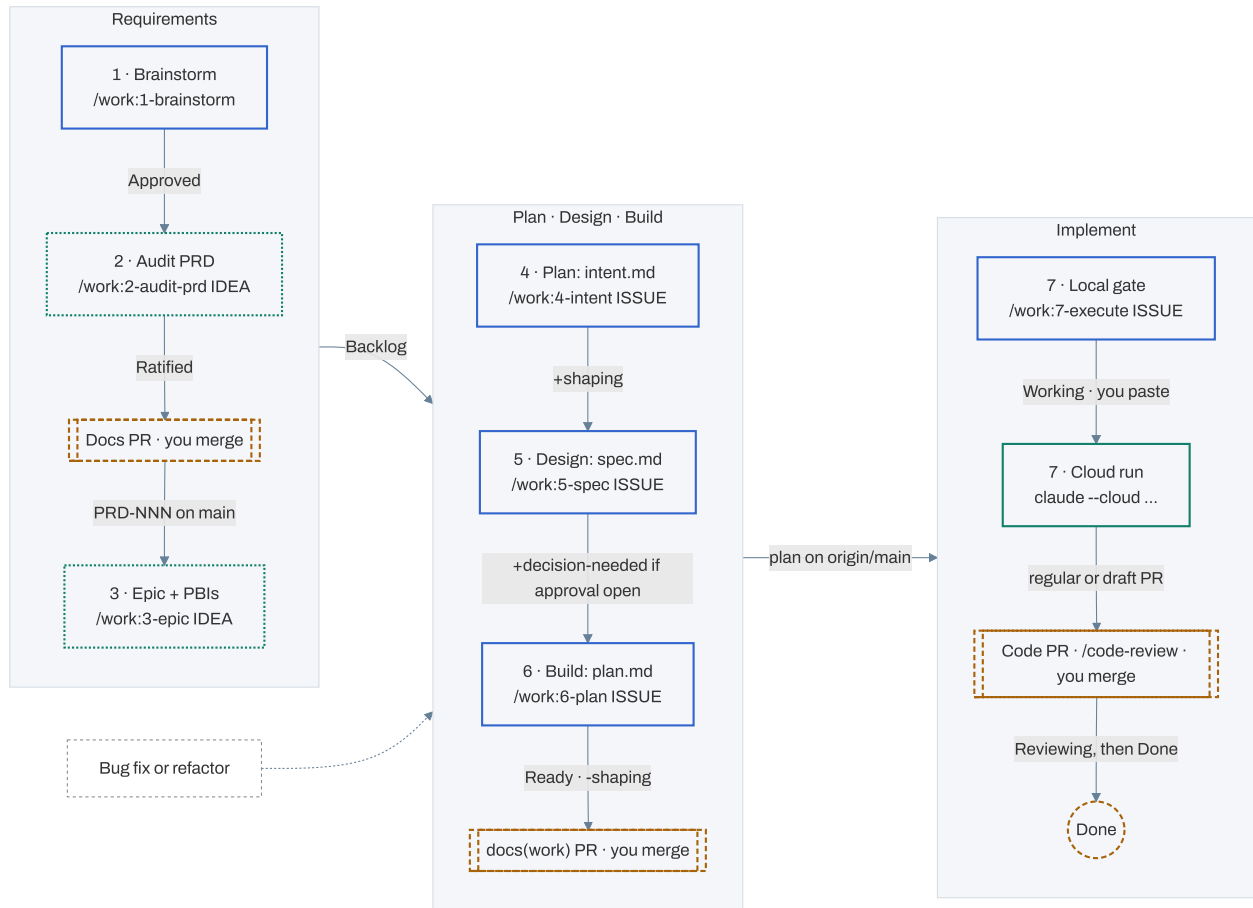
Backlog PBI filed for a requirement /work:3-epic	Backlog Work folder open, plan not approved +shaping · /work:4-intent	Ready Plan approved /work:6-plan
Working Cloud run launched /work:7-execute (local)	Reviewing · you Code PR open gh project item-edit	Done · you PR merged gh project item-edit

shaping · work folder open, plan.md · not approved · decision-needed

spec approved but an approval item is unticked, no code until you decide

The whole flow

Three pull requests carry a requirement to `main` : the docs PR from step 2, the `docs(work)` PR that lands the approved intent, spec and plan before a cloud run, and the code PR from step 7. A bug fix or refactor skips the Requirements phase and starts at step 4.



Step by step

<idea#> is an idea number from docs/work/brainstorming/. <issue#> is a PBI issue number. Start every step in a new session; each one reads only the artifact before it, never the conversation that wrote it.

REQUIREMENTS steps 1–3 · decide what to build and put it on the board

1

Brainstorm the idea

Local

Explore an idea until you can decide whether it should exist, and in what shape.

CLAUDE /work:1-brainstorm "<topic>"

RESUME /work:1-brainstorm <idea#>

SUPERPOWERS superpowers:brainstorming

ASKS Always your 2–3 sentence description of the idea, then batched rounds of questions until nothing is open. The topic is optional.

READS docs/PRODUCT.md , docs/PRD.md , earlier ideas, and reports any overlapping PRD-NNN first

WRITES docs/work/brainstorming/<NNN>-<slug>.md on branch docs/idea-<NNN>-<slug> , pushed so a cloud session can see it. NNN counts across main and open idea branches.

GATE ■ Every open question answered or deferred to step 2. You answer Approved, Rejected, or Not yet.

BOARD No change.

Audit the PRD

● Local

● or cloud

● You merge

Turn an approved idea into requirements: new `PRD-NNN` , changes to existing ones, or both.

CLAUDE

```
/work:2-audit-prd <idea#>
```

TERMINAL

```
claude --cloud "/work:2-audit-prd <idea#>"
```

SKILLS

`/doc-audit` mandatory, every run

AUDIT

Covered · changed · new · conflicts with the out-of-scope list or product anti-patterns · release fit. An idea for a later release goes on the `PRODUCT.md` roadmap and gets no `PRD-NNN` .

GRANULARITY

One `PRD-NNN` is one testable capability, in one capability section, small enough for one work item, with one acceptance path. Prerequisites are numbered and listed first.

WRITES

`PRODUCT.md`, then `PRD.md` (requirements, stories, acceptance criteria, scope, dependencies), each a diff you approve in its own commit. The idea becomes **Ratified** with its "Ratified as" list.

GATE

■ Idea must be Approved. Docs PR on the idea branch; one approving review; you merge.

CLOUD

Interactive: you answer its questions at claude.ai/code. It never touches the board.

Create the epic and PBIs

Local

or cloud

GitHub

File one product backlog item (PBI) per requirement, under its section's epic, with prerequisites linked as blocked-by.

CLAUDE /work:3-epic <idea#>

OR IDS /work:3-epic PRD-053 PRD-008

READS	origin/main only: the idea's "Ratified as" list and docs/PRD.md . A range like PRD-053..PRD-060 is refused.
WRITES	New: PRD-NNN: <title> (Must) as a sub-issue of the section's epic, found through an existing PBI's parent. Changed: the PBI is updated and commented. Retired: closed as not planned, on your yes.
GATE	■ The idea reads Ratified on main . You approve one table with exact titles and bodies before anything is written.
BOARD	Backlog on the project board. In the cloud, through the board's Auto-add workflow.
REPORT	Issues by epic, blocked-by links, work folders made stale by a changed requirement, and the next unblocked PBI.

What it runs

```
gh issue create --title "PRD-NNN: <title> (Must)" --label <module> --body
"<body>"
gh api repos/{owner}/{repo}/issues/<n> --jq .id
gh api -X POST repos/{owner}/{repo}/issues/<epic#>/sub_issues -F
sub_issue_id=<pbi id>
gh api -X POST repos/{owner}/{repo}/issues/<pbi#>/dependencies/blocked_by
-F issue_id=<prerequisite id>
gh project item-add <project#> --owner <org> --url <issue url> # local
only
```

PLAN · DESIGN · BUILD steps 4–6 · one work folder per PBI

Bug fix or refactor? Skip the Requirements phase and start here. Give /work:4-intent a free description and it files the issue.

4

Plan: state the intent

● Local · fresh session

The problem, stated before anyone proposes a solution: why now, the outcome, what is in this slice and what is deliberately out.

CLAUDE /work:4-intent <issue#>

SUPERPOWERS `superpowers:brainstorming` with the questions batched

READS The PBI issue and the `PRD-NNN` it cites; every constraint cited to a `docs/` file

WRITES `docs/work/<NNN>-<slug>-<PRD-NNN>/intent.md` and the folder `README.md`

GATE ■ Every open question answered or deferred with a reason. `Status:` `Approved` only on your explicit yes.

BOARD Backlog · `+shaping`

5

Design: write the spec

● Local · fresh session

Requirements and design in one pass, with the repo's policy applied while it is written instead of discovered in review.

CLAUDE /work:5-spec <issue#>

UI SLICES Shape the screen before designing it; resolve every visual value to a design token

READS `intent.md` only, then the requirement, architecture, file-placement, engineering-rule and design-system docs, and the code it touches

WRITES `spec.md` : behaviour, data model and access rules, server surface and input validation, UI, rejected alternatives, tests, downstream doc changes, items needing your approval

GATE ■ Approved on your yes. Any unticked approval item (schema change, new package, auth or permissions change) labels the issue `decision-needed`.

BOARD Unchanged

6

Build: write the plan

Local · fresh session

A dependency-ordered task list a session with no memory of any of this can execute and turn into a PR.

CLAUDE /work:6-plan <issue#>

SUPERPOWERS `superpowers:test-driven-development` every task's `proof` command must fail before the task starts

READS `spec.md` and the out-of-scope list in `intent.md`, then every file the spec touches

WRITES `plan.md`: execution contract, waves, tasks (`depends-on`, `files`, `proof`), branch `feat/<slug>`, risks, an empty Deviations section

GATE ■ Spec approved and every approval item ticked, then your yes on the plan.

BOARD **Ready** · `-shaping` `-decision-needed`

Then land the folder on main.

Open a `docs(work)` PR with the approved `intent.md`, `spec.md` and `plan.md`; review; you merge. The cloud session clones `origin/main`, so a plan that is not merged is a plan it never sees.

IMPLEMENT step 7 · build it in the cloud

7

Implement: execute the plan

Local

You

Cloud

One command, two modes. Locally it checks the gates and hands you the launch line. Inside the cloud session it builds the plan unattended and opens the PR.

CLAUDE /work:7-execute <issue#>

TERMINAL `claude --cloud "/work:7-execute <NNN>-<slug>-<PRD-NNN>"`

1 · LOCAL GATE

- On `main` , `HEAD` = `origin/main` , folder clean
- `plan.md` Approved
- Plan branch not on origin yet
- Card reads Ready
- A default cloud environment is configured
- Moves the card to **Working**, prints the launch line, stops

2 · YOU LAUNCH

- Paste the line into a standalone terminal at the repo root, still on `main`
- `claude --cloud` needs an interactive terminal, so no Claude session can start it for you
- Follow the run at `claude.ai/code` or in the Claude app

3 · CLOUD RUN

- Preflight: install dependencies, start local services
- Waves in parallel via `superpowers:dispatching-parallel-agents`
- A task is done when its `proof` passes
- `superpowers:verification-before-completion`
- before the PR
- Ship gate: `lint · typecheck · format:check · test`
- Pushes `feat/<slug>` , opens the PR, reports

TESTING

Tests run against local services inside the cloud VM, never against a shared or hosted environment. CI runs the same gate again on the PR.

WHEN BLOCKED

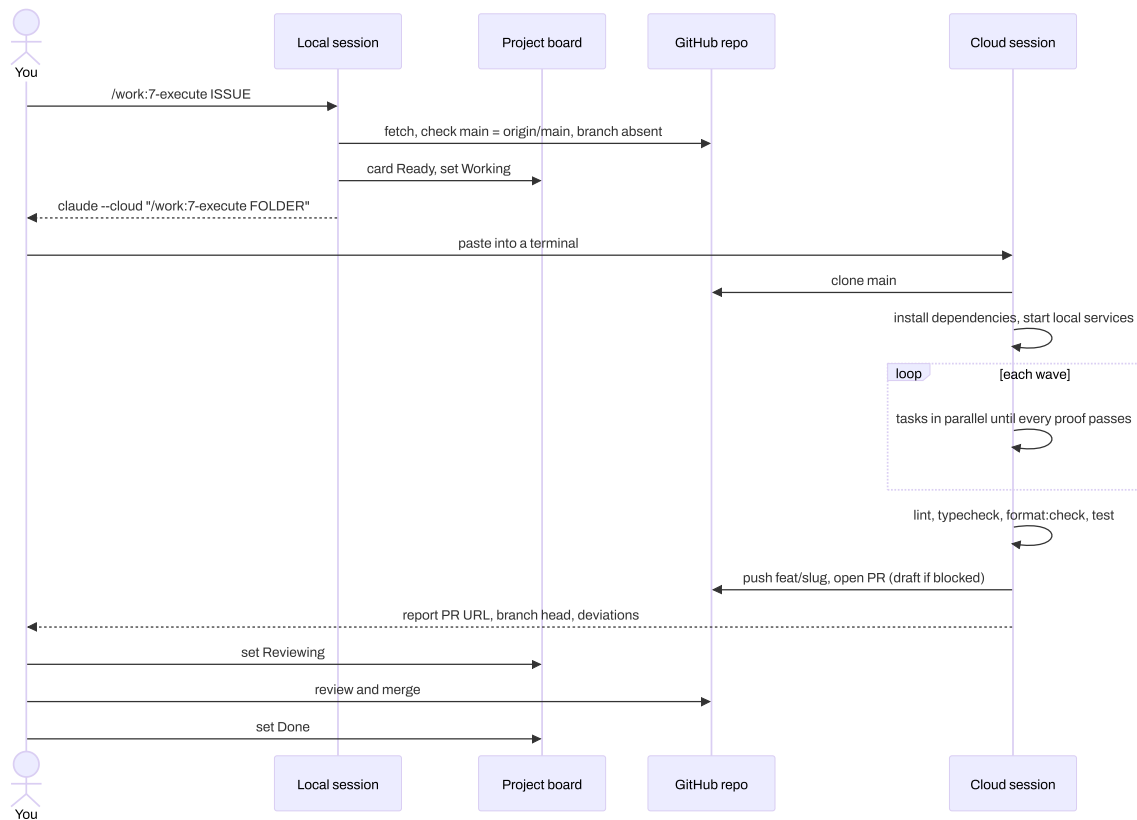
■ The cloud session never asks. A contradiction goes into `plan.md` `$Deviations` and the PR opens as a **draft** titled with what blocked. A failed ship gate also means a draft PR.

NEVER

Moves cards (the cloud can't reach the project board), touches a hosted environment, pushes to `main` , force-pushes, or merges.

Step 7 up close

Who does what, in order, between your machine, the board, GitHub and the cloud session.



After the PR opens

Everything from here is yours. The agent's job ended at the PR.

1. Move the card to Reviewing.

TERMINAL

```
gh project item-edit --id <item id> --project-id <project id>
--field-id <status field id> --single-select-option-id
<Reviewing option id>
```

- Review it with Claude.** `/code-review <PR#>` , then your self-review checklist in the PR description.
- Draft PR or a Deviation?** Open a local session with `superpowers:systematic-debugging` . Fix the plan or the code; never improvise past a wrong plan.
- Review comments.** Work through them with `superpowers:receiving-code-review` . Push first, then ask for review, because a push after approval dismisses it.
- Merge.** One approving review from someone who is not the author, every thread resolved, squash or rebase. You merge; agents never do.
- Schema change in the PR?** Apply it to the hosted environment after merge, from your machine.

7. Move the card to Done.

Rules that hold at every step

One step, one fresh session.

A step reads only the previous artifact. If the file doesn't carry it, it doesn't exist yet.

Only you approve.

`Status: Approved` is set on your explicit yes, and the next step refuses to run on a Draft.

Requirements before issues.

Step 3 reads only `main`, so nothing is filed until its PRD change has merged.

Every board write prompts.

Nothing reaches GitHub issues or the board without an approval click.

Main is protected.

No push to `main`, no force-push, no merge or close by an agent. A hook enforces it.

Plan wrong? Stop.

Log it in §Deviations and open a draft PR. A plan wrong about one thing is evidence.

Cloud sessions.

Steps 2–3 can run interactively in the cloud; step 7 runs there unattended. The cloud cannot reach the board, so issues arrive through Auto-add.

Hosted credentials stay local.

Anything that needs a login to a hosted environment runs on your machine, not in the cloud.

Commands live in `.claude/commands/work/`. Folder rules: `docs/work/README.md` and `docs/work/brainstorming/README.md`.